

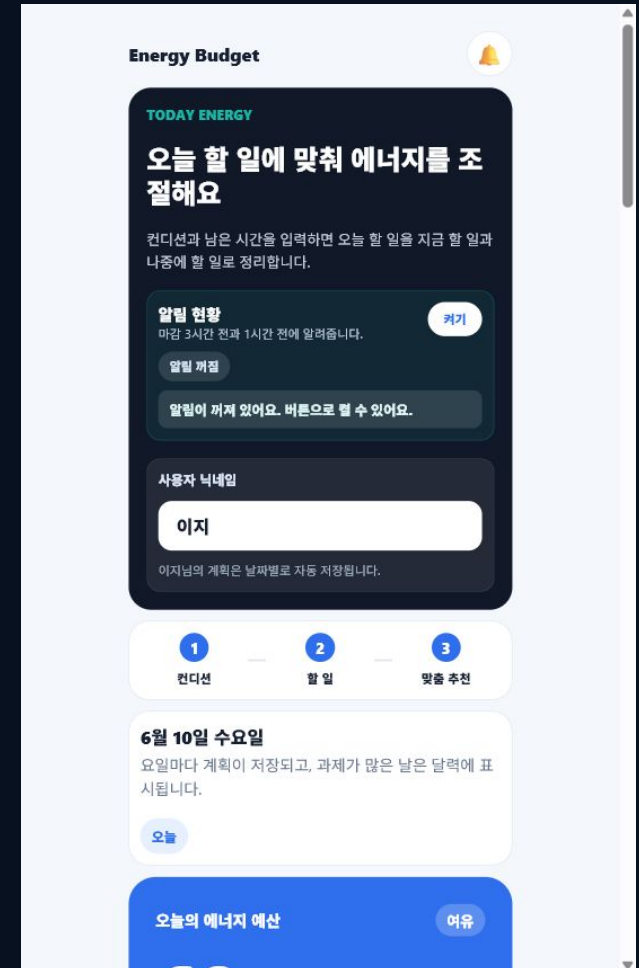
FINAL PRESENTATION

Energy Budget

오늘 할 일에 맞춰 에너지를 조절하는
모바일 앱

2024136064 정원식

React Native + Expo + TypeScript



평가 기준 대응표

평가 영역	PDF에서 보여주는 내용	발표에서 말할 핵심
발표 체계성 10점	비전, 문제 정의, WBS, 기술, 구현, 활용 흐름	왜 만들었고 어떻게 만들었는지 순서대로 설명
질의응답 5점	ADR 4개, 앱 구조, 개발 환경, 빌드/배포 단계	선택 이유를 기록 기준으로 답변
개발자 기본소양 10점	기술 설명, 아키텍처, 시행착오, 테스트, 코드 품질	읽지 않고 이해한 내용으로 설명
시연 데모 5점	30초 데모 순서, 앱 화면, 핵심 기능, 임팩트 포인트	사용자가 실제로 쓰는 장면처럼 보여주기

발표 전략: 점수표의 모든 질문을 슬라이드 안에 숨기지 않고, 제목과 문장으로 직접 드러낸다.
 그래서 발표자는 '비전 - 문제 - 해결 - 구현 - 검증 - 활용' 순서만 따라가도 평가 항목을 자연스럽게 충족한다.

비전: 시간을 채우는 앱이 아니라, 나를 지키는 계획 앱

Energy Budget은 사람들의 하루 계획을 시간 중심에서 에너지 중심으로 바꾸어, 무리하지 않아도 꾸준히 성장할 수 있는 지속 가능한 일상을 만드는 앱이다.

- 목표: 사용자가 오늘 감당 가능한 일을 먼저 알고, 무리한 계획을 줄이게 한다.
- 영향: 계획 실패를 자책으로 끝내지 않고, 컨디션 기반 조절로 다시 시작하게 만든다.
- 가치: 더 많이 시키는 도구가 아니라 오래 지속할 수 있는 하루를 설계하는 도구다.
- 미래지향성: 기록이 쌓이면 요일별 에너지 패턴과 과제 과부하를 예측하는 앱으로 확장한다.

문제 정의: 계획 실패는 의지 부족이 아니라 과부하 설계 문제

기존 투두앱의 한계

- 할 일을 많이 담는 데 집중한다.
- 시간은 보지만 피로도와 기분은 잘 보지 않는다.
- 계획이 밀리면 사용자는 정리보다 부담을 먼저 느낀다.

Energy Budget의 관점

- 시간보다 먼저 오늘의 에너지를 계산한다.
- 피로도, 기분, 남은 시간을 계획의 첫 입력값으로 둔다.
- 감당 가능한 일과 미뤄도 되는 일을 나눠 과부하를 줄인다.

공감 문장: 시간은 남았는데 몸이 안 따라와서 계획이 무너지는 순간을 해결한다.

제품 흐름: 오늘 상태를 입력하면 할 일을 다시 정리한다

1

INPUT

피로도, 기분, 남은 시간, 닉네임, 날짜별 계획을 입력

2

BUDGET

오늘 감당 가능한 에너지 예산을 계산

3

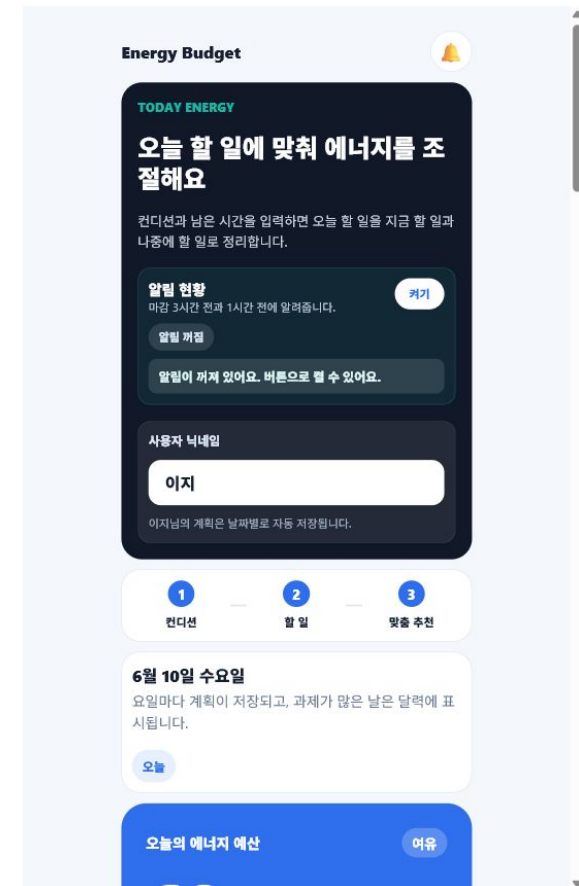
RECOMMEND

지금 하면 좋은 일과 나중에 미룰 일을 분리

4

REMIND

마감 3시간 전과 1시간 전 알림 현황을 표시

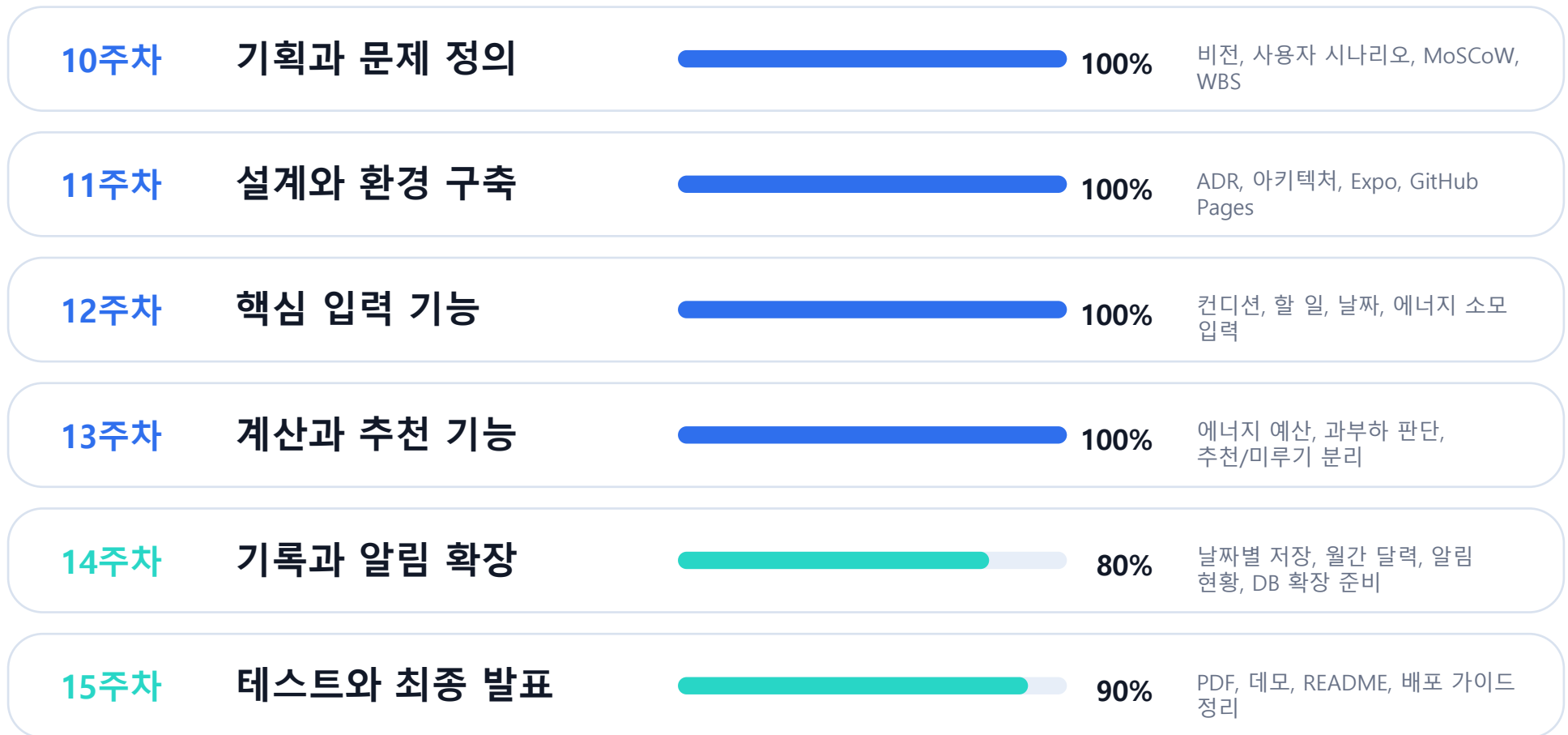


30초 시연 데모: 사용자가 실제로 쓰는 흐름으로 보여주기

순서	보여줄 행동	말할 포인트
1	닉네임 입력	개인 계획이 날짜별로 저장되는 앱이라는 점
2	피로도과 남은 시간 조절	오늘의 컨디션이 계획의 기준이 됨
3	할 일 추가: 시간, 날짜, 마감 시간, 에너지 소모 입력	일마다 에너지 비용을 붙임
4	자동 추천 선택	앱이 지금 할 일과 미룰 일을 나눔
5	달력과 알림 현황 확인	과제가 많은 날과 마감 알림을 확인

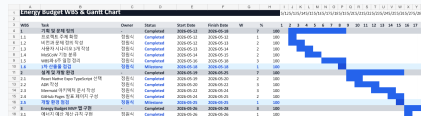
시연 임팩트: '할 일을 더 넣는 앱'이 아니라 '오늘 못 해도 괜찮은 일을 골라주는 앱'으로 보여준다.

WBS 진행 현황: MVP 핵심 경험 완료, 저장과 리포트 확장 중



FM WBS: 산출물 중심으로 쪼개고, 완료 기준을 붙였다

WBS	작업	Owner	상태	산출물/완료 기준
1	기획	정원식	완료	비전, 문제 정의, 사용자 시나리오, MoSCoW
2	설계	정원식	완료	ADR 4개, Mermaid 아키텍처, 디렉토리 구조
3	핵심 입력	정원식	완료	피로도, 기분, 남은 시간, 할 일, 날짜, 에너지 소모
4	추천 로직	정원식	완료	에너지 예산 계산, 지금 할 일/미룰 일 분리
5	기록/알림	정원식	진행	날짜별 저장, 월간 보기, 마감 알림 현황
6	검증/발표	정원식	진행	typecheck, 데모 시나리오, 최종 평가 PDF



적용 기술: 모바일 MVP를 빠르게 만들고 설명 가능하게 구성

React Native

모바일 앱 UI를 구성. 웹 미리보기로 발표 중 빠른 확인 가능.

Expo

복잡한 네이티브 설정 없이 실행, Android/Web export 검증 가능.

TypeScript

데이터 타입을 명확히 해서 입력값과 계산 로직 오류를 줄임.

AsyncStorage

현재 MVP의 닉네임, 날짜별 계획, 알림 설정을 로컬 저장.

Notification API

마감 3시간 전/1시간 전 알림 현황을 앱 내부에서 표시.

GitHub Pages

발표자료와 WBS를 URL로 바로 열 수 있게 준비.

Mermaid

아키텍처 흐름을 텍스트 기반 다이어그램으로 표현.

아키텍처: 화면과 계산 규칙을 분리해서 설명 가능한 구조로 만들었다

사용자

피로도, 기분, 시간, 할 일을 입력

presentation

HomeScreen, 모바일 UI, 입력 폼

application

화면 흐름과 view model

domain

에너지 예산 계산, 추천 규칙

data

AsyncStorage 저장, Repository

디렉토리 구조

- src/presentation - 사용자가 보는 화면
- src/application - 화면 흐름과 use case
- src/domain - 핵심 엔티티와 계산 규칙
- src/data - 저장소와 Repository
- docs - 발표, 배포, 테스트 문서
- .planning - 비전, 요구사항, WBS, ADR

구현 설명: 감정적인 문제를 계산 가능한 흐름으로 바꿨다

상태 입력

피로도, 기분, 남은 시간, 닉네임, 선택 날짜를 React 상태로 관리한다.

도메인 계산

calculateEnergyBudget이 컨디션 값을 에너지 예산으로 바꾸고, 총 소모량과 비교한다.

추천 분리

할 일의 에너지 소모가 예산 안에 들어오면 지금 할 일, 넘치면 미루기 후보로 분리한다.

수정 가능성

할 일 추가 후 제목, 카테고리, 날짜, 마감 시간, 에너지 소모를 수정할 수 있게 했다.

요일 저장

날짜별 계획을 로컬 저장해 월간 달력에서 과제가 많은 날을 확인할 수 있게 했다.

알림 현황

마감까지 3시간/1시간 남은 일을 앱 상단에서 먼저 보여준다.

ADR 최소 3개 이상: 선택 이유를 기록으로 답변한다

ADR	결정	이유	질문 답변 포인트
0001	React Native + Expo	모바일 사용성이 중요하고 6주 안에 MVP 구현 가능	왜 모바일인가? 생활 속에서 자주 확인해야 하기 때문
0002	React 기본 상태 + 도메인 분리	초기 화면 수가 적고 계산 규칙을 따로 설명하기 쉬움	왜 Redux가 아닌가? MVP에는 과함
0003	초기 로컬 저장	계정/서버 없이도 핵심 가치 검증 가능	DB는 다음 확장, 현재는 AsyncStorage 중심
0004	Energy Budget 주제	실생활 사용성과 발표/포트폴리오 설명력이 높음	왜 이 주제인가? 공감 가능한 문제와 구현 범위가 맞음

Q&A 원칙: 외운 답보다 ADR의 '상황 - 결정 - 이유 - 결과' 순서로 답한다.

개발 환경, 빌드, 배포: 실행 가능한 산출물로 검증

구분	명령/산출물	의미
설치	npm install	프로젝트 의존성을 내려받아 같은 환경을 만든다.
개발 실행	npm run start / npm run web	Expo 개발 서버를 열고 모바일 UI를 확인한다.
정적 검사	npm run typecheck	TypeScript 기준으로 코드 오류를 먼저 확인한다.
Android export	npm run export:android	Android용 번들 생성 가능성을 검증한다.
Web export	npm run export:web	PC 브라우저 백업 실행과 Pages 배포 자료를 만든다.
GitHub Pages	발표 PDF + WBS URL	발표 시작 지연 없이 PC에서 바로 열 수 있게 한다.

빌드는 소스코드를 실행 가능한 산출물로 만드는 과정이고, 배포는 다른 사람이 접근할 수 있는 위치에 올리는 과정이다.

개발자 기본소양: 이해, 개선, 검증을 함께 보여준다

시행착오

처음에는 기능을 많이 넣으려 했지만, WBS와 MoSCoW로 MVP 범위를 줄였다.

성능 최적화

계산 로직은 입력이 바뀔 때만 다시 계산하고, 화면은 모바일 폭에 맞춰 과한 여백을 줄였다.

코드 품질

TypeScript 타입, 도메인 규칙 분리, Repository 구조로 화면과 계산을 분리했다.

단위 테스트 관점

energyBudgetRules의 예산 계산, 총 소모량, 과부하 판단을 독립적으로 검증 대상으로 잡았다.

통합 테스트 관점

컨디션 입력 - 할 일 추가 - 추천 분리 - 저장 - 달력 확인 흐름을 사용자 시나리오로 확인했다.

설치 가이드

README, docs/setup.md, docs/deploy.md에 실행과 배포 방법을 정리했다.

예상 질문 답변: ADR 기준으로 짧고 정확하게

Q1. DB도 만들었나요?

현재 MVP는 AsyncStorage로 날짜별 계획을 저장합니다. SQLite는 다음 단계이며, ADR-0003에서 서버 없이 먼저 핵심 가치를 검증하기로 기록했습니다.

Q2. 앱 구조는 어떻게 되나요?

presentation은 화면, application은 흐름, domain은 계산 규칙, data는 저장 역할입니다. 화면과 계산을 분리해서 수정과 설명이 쉽습니다.

Q3. 왜 Expo를 썼나요?

6주 안에 모바일 MVP를 만들고 Android/Web export를 빠르게 검증하기 위해서입니다. 네이티브 기능이 많지 않아 Expo가 적합했습니다.

Q4. 빌드와 배포는 어떻게 했나요?

typecheck로 코드 오류를 확인하고 Expo export로 산출물을 만든 뒤, 발표 자료와 WBS는 GitHub Pages에서 URL로 확인하게 했습니다.

Q5. 앞으로 어떻게 활용하나요?

과제 많은 요일과 에너지 소모를 기록해 나중에는 주간 리포트와 SQLite 저장으로 확장할 계획입니다.

감사합니다

Q&A

Energy Budget은 더 많이 하게 만드는 앱이 아니라, 오늘의 나를 기준으로 계획을 다시 세우게 돕는 앱입니다.